

Capstone Project

Agentic Text-to-SQL Small Language Model with Graph Schema & RAG Pipeline

Architecture & Technical Design Document

Document Version	1.0
Date	March 2026
Author	[Student Name]
Mentor	[Mentor Name]
Program	[Program Name]
Classification	Academic / Internal

Table of Contents

1. Problem Statement
2. Proposed Solution
3. Solution Approach — Step-by-Step
4. High-Level Architecture
5. Low-Level Architecture
 - 5.1 Schema Ingestion Layer
 - 5.2 Graph Schema Layer (Neo4j)
 - 5.3 RAG Pipeline
 - 5.4 Fine-Tuning Pipeline
 - 5.5 Agentic Orchestration Layer
 - 5.6 Inference Engine
 - 5.7 SQL Validation & Self-Correction
 - 5.8 Evaluation & Feedback Loop
6. Technology Stack
7. Data Flow — End-to-End Walkthrough
8. Accuracy Strategy (70%+ Target)
9. Deployment Architecture
10. Project Timeline & Milestones
11. Risks & Mitigation
12. References

1. Problem Statement

Organizations today store vast amounts of critical business data in relational databases (SQL Server, PostgreSQL, MySQL, Oracle). However, extracting insights from these databases requires writing SQL queries — a skill that most business users, analysts, and decision-makers do not possess. This creates a significant bottleneck where technical teams become the gatekeepers of data access, slowing down decision-making and increasing operational costs.

Key Challenges:

- **SQL Expertise Gap:** Business users cannot write SQL queries, especially complex multi-table JOINS, subqueries, aggregations, and window functions. They depend entirely on data engineering teams.
- **Schema Complexity:** Enterprise databases have hundreds of tables with intricate foreign key relationships. Even experienced developers struggle to identify correct join paths across 5-10 tables.
- **Schema Variability:** Every organization has a unique database schema. A generic text-to-SQL tool trained on public benchmarks (Spider, WikiSQL) fails on production schemas because it has never seen the specific table and column names, relationships, or business terminology.
- **Accuracy Requirements:** Generated SQL must be syntactically correct AND semantically accurate — returning the right data. A wrong query is worse than no query because users may make decisions based on incorrect results.
- **Complex Join Generation:** The hardest category of text-to-SQL is multi-table JOIN queries. Users ask questions like 'show me revenue by product category per region' which requires discovering and chaining 3-5 tables through foreign key relationships that the user never explicitly mentions.
- **Cost of Large Models:** Commercial LLMs (GPT-4, Claude) can handle text-to-SQL but are expensive for high-volume production use, introduce latency, and require sending proprietary schema data to third-party APIs — a security concern for many enterprises.

Core Problem: There is no affordable, self-hosted, schema-adaptive system that can take a natural language question and generate accurate SQL queries — including complex multi-table JOINS — for any arbitrary database schema, with 70%+ accuracy.

2. Proposed Solution

We propose building an **Agentic Text-to-SQL Small Language Model (SLM) System** that combines four cutting-edge AI/ML techniques into a unified, self-hosted pipeline capable of generating accurate SQL from natural language for any database schema.

Solution Pillars:

Graph-Based Schema Representation (Neo4j)

Instead of treating database schemas as flat text, we model them as a **property graph** where tables are nodes, foreign keys are directed edges, and columns are node properties. This graph structure enables automatic discovery of multi-hop join paths using BFS/Dijkstra traversal. When a user asks about 'revenue by product per user,' the system traverses: users → orders → order_items → products, discovering the exact JOIN chain automatically — even across bridge tables the user never mentioned.

RAG Pipeline (Retrieval-Augmented Generation)

For large schemas (100+ tables), the model cannot fit all table definitions into its context window. RAG solves this by embedding table and column descriptions into a vector store (FAISS/ChromaDB), then retrieving only the relevant subset at query time. Combined with the graph join planner, this dual-retrieval approach fetches both the right tables AND the correct join paths.

Schema-Specific Fine-Tuning (QLoRA)

We fine-tune a Small Language Model (CodeLlama-7B or Phi-3-mini) using QLoRA (4-bit quantized LoRA) on schema-conditioned NL-to-SQL pairs. This makes the model expert at your specific database's patterns, terminology, and query structures — while keeping GPU requirements under 10GB VRAM, runnable on a single consumer GPU.

Agentic Self-Correction (LangGraph + ReAct)

Instead of a single-pass 'generate and hope' approach, an **AI agent orchestrator** (LangGraph) runs a ReAct (Reason-Act-Observe) loop: the agent plans the query, generates SQL, validates it (syntax check + sandbox execution), and if it fails, feeds the error back into the prompt for self-correction — up to 3 retries. This iterative approach pushes accuracy from ~70% to 80-85%+.

Key Innovation: The combination of graph schema (for join discovery) + RAG (for schema retrieval) + fine-tuned SLM (for SQL generation) + agentic self-correction (for accuracy) creates a system that is generic enough to adapt to any schema yet accurate enough for production use.

3. Solution Approach — Step-by-Step

This section provides the complete step-by-step implementation plan, ordered by execution sequence. Each step builds on the previous one.

Step 1: Schema Ingestion & Normalization

Import the database schema from two sources: (a) an **Excel file** containing table names, column names, data types, primary keys, foreign keys, and business descriptions — parsed using openpyxl/pandas; (b) a **live SQL database connection** via SQLAlchemy that reads INFORMATION_SCHEMA or uses the inspector API to extract schema metadata automatically. Both sources are normalized into a canonical DDL-like format (CREATE TABLE statements with annotated comments) that serves as the universal input for all downstream components. This step also extracts business glossary terms (e.g., 'revenue' maps to orders.total) from the Excel descriptions.

Step 2: Build the Graph Schema (Neo4j Property Graph)

Convert the normalized schema into a **Neo4j property graph**. Each table becomes a node with properties: table_name, column_list (with types), primary_key, row_count estimate, and business description. Each foreign key relationship becomes a directed edge with properties: FK column name, referenced column, cardinality (1:1, 1:N, M:N), and a usage_weight (initially 1.0, updated based on query frequency). For schemas imported from Excel, FK relationships are explicitly defined in the spreadsheet. For live DB connections, they are auto-detected from INFORMATION_SCHEMA.KEY_COLUMN_USAGE. The graph also stores derived edges for implicit relationships (e.g., two tables sharing a common column name pattern like 'user_id').

Step 3: Create Schema Embeddings (Vector Store)

For each table, concatenate: table_name + column_names + column_types + business_description into a text chunk. Embed each chunk using **sentence-transformers/all-MiniLM-L6-v2** (384-dim vectors, fast and lightweight). Store embeddings in **FAISS** (for local/offline) or **ChromaDB** (for persistent storage with metadata filtering). Also embed individual column descriptions for fine-grained retrieval when the user mentions specific fields. Index metadata (table_name, db_name, schema_version) alongside each vector for filtered retrieval.

Step 4: Generate Training Data (NL-SQL Pairs)

Create schema-conditioned training pairs in the format: {schema_context, join_path, nl_question, sql_query}. Sources: (a) **Public benchmarks** — Spider, BIRD, WikiSQL adapted to your schema structure; (b) **Template-based generation** — walk random paths in the graph (2-4 hops), generate JOIN queries, then use a teacher LLM (Claude/GPT-4) to produce diverse NL phrasings for each query; (c) **Historical query logs** — if available, mine past SQL queries and pair them with NL descriptions. Target: 5,000-10,000 high-quality pairs covering single-table, 2-table JOIN, and 3-5 table complex JOIN scenarios. Split: 80% train, 10% validation, 10% test.

Step 5: Fine-Tune the Base SLM (QLoRA)

Select a base model: **CodeLlama-7B-Instruct** (best accuracy ceiling) or **Phi-3-mini-4k-instruct (3.8B)** (faster, less VRAM). Fine-tune using **QLoRA**: load the base model in 4-bit quantization (NF4 via bitsandbytes), attach LoRA adapters (rank=16, alpha=32) to attention layers (q_proj, k_proj, v_proj, o_proj), and train using Hugging Face's **SFTTrainer** from the TRL library. Hyperparameters: learning_rate=2e-4, batch_size=4, gradient_accumulation=4, epochs=3, warmup_ratio=0.05. Training takes ~4-8 hours on a single A100/RTX 4090 (or ~12 hours on RTX 3090). VRAM requirement: 8-12GB.

Step 6: Build the RAG Retrieval Pipeline

At inference time, the RAG pipeline runs two parallel retrieval paths: (a) **Vector retrieval** — embed the user's NL question, search FAISS for top-k (k=5-10) most similar table descriptions; (b) **Graph join planner** — take entity-linked table names from the NL preprocessor, query Neo4j using Cypher shortestPath() to find FK chains connecting them. A **cross-encoder reranker** (ms-marco-MiniLM-L-6-v2) scores the combined context chunks for relevance. The **context assembler** merges: relevant table DDLs + join ON conditions + column metadata + sample values into a structured prompt template.

Step 7: Build the Agentic Orchestration Layer (LangGraph)

Wrap the entire pipeline in a **LangGraph state machine** with three agent nodes: (a) **Planner agent** — decomposes complex NL queries, determines if single-table or multi-join; (b) **SQL generator agent** — calls RAG tools, assembles prompt, runs the fine-tuned SLM to produce SQL; (c) **Validator agent** — lints SQL (sqlglot AST), checks column/table existence against schema, executes on a read-only sandbox DB, inspects results. Conditional edges: if validation passes → return SQL; if fails → route back to generator with error context (max 3 retries). Agent tools are exposed via **MCP (Model Context Protocol)** for interoperability. Memory: conversation buffer (short-term), successful query cache (long-term), error pattern store (episodic).

Step 8: Evaluation, Testing & Feedback Loop

Evaluate on held-out test set using: **Execution Accuracy (EX)** — does the query return correct results compared to gold-standard SQL? **Exact Match (EM)** — does the SQL string match? **Valid SQL Rate (VSR)** — percentage of syntactically valid queries. Target: EX > 70%, VSR > 95%. Set up an **active learning loop**: log all user queries and corrections, periodically re-train LoRA adapters on the accumulated corrected pairs. Use **LangSmith** for agent trace observability — inspect each step the agent took, which tools it called, and where failures occurred.

Step 9: Deploy & Serve

Package the system: **vLLM** serves the fine-tuned SLM with LoRA adapters (GPU) or **llama.cpp / ONNX Runtime** for CPU deployment. **FastAPI** exposes REST endpoints (/upload-schema, /generate-sql, /validate-sql). **Streamlit** provides a chat UI for business users. **NeMo Guardrails** prevents harmful SQL (DROP, DELETE, UPDATE). Everything runs in **Docker containers** orchestrated with docker-compose (dev) or Kubernetes (production). **Prometheus + Grafana** monitor latency, accuracy drift, and query volume.

4. High-Level Architecture

The system is organized into six logical layers, each responsible for a distinct phase of the text-to-SQL pipeline. Data flows top-down from user input to SQL output, with a feedback loop returning corrections from the evaluation layer back to the fine-tuning layer.

Layer	Components	Purpose
1. Schema Ingestion	Excel Importer, SQL Connector, Schema Normalizer	Import & standardize schema from any source
2. Schema Representation	Neo4j Graph, FAISS Vectors, Metadata Catalog	Store schema as graph + embeddings for retrieval & join discovery
3. Fine-Tuning	QLoRA Trainer, Training Data Generator, Base SLM	Adapt SLM to project-specific schema & SQL patterns
4. Agentic Orchestration	LangGraph State Machine, Planner / Generator / Validator	ReAct loop: reason, generate, validate, self-correct
5. Inference & Serving	vLLM/ONNX, FastAPI, Streamlit, Guardrails	Serve queries via API & chat UI with safety guardrails
6. Evaluation & Feedback	EX/EM Metrics, Sandbox Executor, Active Learning Loop	Measure accuracy, collect corrections, re-train

Figure 1: High-level architecture layers. Data flows from Layer 1 (ingestion) through Layer 5 (serving). Layer 6 feeds corrections back to Layer 3 for continuous improvement.

Data Flow Summary:

- 1 User uploads Excel schema OR system connects to live database
- 2 Schema is normalized to canonical DDL and loaded into Neo4j graph + FAISS vector store
- 3 Training data is generated from the schema graph and used to fine-tune the SLM via QLoRA
- 4 At inference: user types NL question → agent plans → RAG retrieves schema context + join paths → SLM generates SQL → validator checks → agent self-corrects if needed → SQL returned
- 5 Evaluation metrics track accuracy; failed queries enter the active learning loop for re-training

5. Low-Level Architecture — Component Details

5.1 Schema Ingestion Layer

This layer is responsible for importing database schemas from heterogeneous sources and converting them into a single canonical format that all downstream components can consume.

Excel Schema Importer

Reads .xlsx/.csv files using **openpyxl** or **pandas**. Expected columns: `table_name`, `column_name`, `data_type`, `is_primary_key`, `is_foreign_key`, `fk_references` (format: 'schema.table.column'), `nullable`, `description`. The importer validates data types against a supported set, flags missing PKs, and auto-detects FK relationships from naming conventions (e.g., 'user_id' → users.id) when explicit FK references are not provided. Output: list of `TableSchema` objects with validated relationships.

SQL Database Connector

Uses **SQLAlchemy**'s inspector API to connect to live databases. Supports: PostgreSQL, MySQL, SQL Server, Oracle, SQLite. Extracts: table names, column definitions (name, type, nullable, default), primary keys, foreign keys with referenced table/column, indexes, and unique constraints. Connection string is provided via environment variable or configuration file. Auto-discovers all schemas in the database and lets the user select which schemas to include.

Schema Normalizer

Converts both Excel-imported and SQL-extracted schemas into a canonical DDL format: CREATE TABLE statements with column type annotations, PRIMARY KEY constraints, FOREIGN KEY constraints with ON DELETE/UPDATE behavior, and inline comments containing business descriptions. Also generates a **business glossary** mapping (e.g., 'revenue' → SUM(orders.total), 'active users' → users WHERE status='active') extracted from the description fields. Output format is database-agnostic — the same normalized schema works whether the source was Excel or a live PostgreSQL database.

Expected Excel Schema Format:

Column	Type	Example	Required
table_name	string	orders	Yes
column_name	string	user_id	Yes
data_type	string	INTEGER	Yes
is_primary_key	boolean	FALSE	Yes
is_foreign_key	boolean	TRUE	No
fk_references	string	public.users.id	If FK

Column	Type	Example	Required
nullable	boolean	FALSE	No
description	string	ID of ordering user	Recommended

5.2 Graph Schema Layer (Neo4j)

The graph schema is the **core innovation** that enables complex multi-table JOIN generation. It models the database schema as a property graph where relationships between tables are first-class citizens — not afterthoughts buried in metadata.

Why a Graph? — The Join Discovery Problem

When a user asks 'show total revenue by product category per region,' the system must discover that this requires: **users JOIN orders ON users.id = orders.user_id JOIN order_items ON orders.id = order_items.order_id JOIN products ON order_items.product_id = products.id JOIN categories ON products.category_id = categories.id**. That is a 5-table, 4-hop join chain. The user mentioned none of these intermediate tables. A flat schema list cannot discover this path. A graph can — using BFS traversal from the 'users' node to the 'categories' node, following FK edges, the system automatically constructs the complete JOIN chain with all ON conditions.

Element	Representation	Properties Stored
Table	Node (:Table)	name, column_list, pk_columns, row_count, description
Column	Node (:Column)	name, data_type, nullable, is_pk, is_fk, description
Foreign Key	Edge [:FK]	from_col, to_col, cardinality, usage_weight, join_type
Implicit Rel.	Edge [:IMPLICIT]	confidence_score, matched_on (naming convention)
HAS_COLUMN	Edge [:HAS_COLUMN]	ordinal_position

Join Path Discovery Algorithm

Uses Neo4j Cypher's **shortestPath()** with a max depth of 5 hops: `MATCH path = shortestPath((a:Table {name: $source})-[:FK*..5]-(b:Table {name: $target})) RETURN path, [r IN relationships(path) | r.from_col + '=' + r.to_col] AS join_conditions`. When multiple paths exist, they are ranked by: (1) shortest hop count, (2) highest cumulative usage_weight (frequently-used joins rank higher), (3) preference for explicit FK edges over implicit edges. The algorithm also handles **self-joins** (same table referenced twice with aliases) and **M:N relationships** through bridge/junction tables.

Lightweight Alternative: For projects that cannot run Neo4j, the graph can be implemented using **NetworkX** (Python in-memory graph library). Trade-off: no persistent storage, no Cypher query language, but zero infrastructure overhead. Suitable for schemas under 200 tables.

5.3 RAG Pipeline

Retrieval-Augmented Generation ensures the SLM sees only the relevant schema context at inference time, enabling it to work with databases containing hundreds of tables without exceeding the model's context window.

Embedding & Indexing (Offline Phase)

For each table, create an embedding text: '[TABLE: orders] Columns: id (INT, PK), user_id (INT, FK→users.id), total (DECIMAL), created_at (TIMESTAMP). Description: Stores customer purchase orders with total amounts.' Embed using sentence-transformers/all-MiniLM-L6-v2 (384 dimensions, ~22M parameters, runs on CPU in <100ms per table). Store in FAISS IndexFlatIP (inner product) for exact search or IndexIVFFlat for approximate search on large schemas (1000+ tables). Also create column-level embeddings for fine-grained retrieval when users mention specific fields.

Dual Retrieval (Online Phase)

Two retrieval paths run in parallel: **(1) Vector retrieval** — embed the user's NL question, search FAISS for top-k=10 most similar table descriptions, return table DDLs and their similarity scores. **(2) Graph join planner** — extract entity-linked table names from the NL preprocessor (e.g., 'revenue' → orders, 'product' → products), query Neo4j for shortest join paths between all extracted table pairs, return JOIN chains with ON conditions. Both results feed into the context assembler.

Context Assembly & Reranking

The assembler merges vector-retrieved tables and graph-discovered join paths into a structured prompt. A **cross-encoder reranker** (ms-marco-MiniLM-L-6-v2) scores each table chunk's relevance to the NL question, removing low-scoring tables that were retrieved by vector similarity but are not actually needed. The final prompt contains: (1) relevant CREATE TABLE DDLs, (2) JOIN paths with ON conditions, (3) column metadata and sample values, (4) business glossary mappings. Token budget: ~2000 tokens for schema context, leaving room for the question and generated SQL.

5.4 Fine-Tuning Pipeline

Fine-tuning adapts the generic base SLM to understand your specific database schema, business terminology, and SQL patterns.

Parameter	Value	Notes
Base Model	CodeLlama-7B-Instruct or Phi-3-mini-4k-instruct	CodeLlama: higher accuracy ceiling Phi-3: faster, less VRAM
Quantization	NF4 (4-bit) via bitsandbytes	Reduces VRAM from 28GB to ~8GB
LoRA Rank (r)	16	Higher = more capacity, more VRAM
LoRA Alpha	32	Scaling factor, typically 2x rank
Target Modules	q_proj, k_proj, v_proj, o_proj	Attention layer projections
Learning Rate	2e-4	With cosine scheduler
Batch Size	4 (effective 16 with gradient accumulation)	gradient_accumulation_steps=4
Epochs	3	Monitor val loss for early stopping
Training Samples	5,000 - 10,000 NL-SQL pairs	Quality > quantity
Training Time	4-8 hours (A100) 12 hours (RTX 3090)	Single GPU sufficient
VRAM Required	8-12 GB	Fits consumer GPUs

Training Data Format

Each training sample is a JSON object with four fields: **schema_context** (the relevant CREATE TABLE statements), **join_path** (discovered FK chain, e.g., 'users.id = orders.user_id AND orders.id = order_items.order_id'), **question** (natural language), **sql** (gold-standard SQL query). The prompt template follows: '[INST] Given the following database schema:\n{schema_context}\nJoin path: {join_path}\n\nGenerate SQL for: {question} [/INST]\n{sql}'. Diverse NL phrasings are generated using a teacher LLM to prevent overfitting to one phrasing style.

5.5 Agentic Orchestration Layer

The agentic layer transforms the system from a single-pass generator into an intelligent, self-correcting pipeline. This is the layer that pushes accuracy from ~70% to 80-85%+.

Orchestrator: LangGraph State Machine

The agent is implemented as a **LangGraph** state graph with three nodes (Planner, Generator, Validator) and conditional edges. State is a TypedDict containing: nl_query, schema_context, join_paths, generated_sql, validation_result, error_message, retry_count. The graph processes: START → Planner → Generator → Validator → (Pass: END, Fail: Generator with error context). Maximum retry_count = 3. Each node has access to tools via function calling. LangGraph's built-in checkpointing enables session persistence and replay for debugging.

Agent	Role	Tools Used	Output
Planner	Decompose NL query, identify tables needed	schema_graph_tool, nl_preprocessor	Query plan + table list
SQL Generator	Retrieve context, generate SQL	rag_retrieval_tool, slm_inference_tool	SQL query string
Validator	Check syntax, schema refs, execute SQL	sql_linter_tool, sql_executor_tool	Pass/Fail + error msg

ReAct Reasoning Pattern (Reason → Act → Observe)

Reason: 'The user asks for revenue by product per user. I need tables: users, orders, order_items, products. This requires 3 JOINS.' → **Act:** Call graph_tool to find join paths, call rag_tool to retrieve DDLs, call slm_tool to generate SQL. → **Observe:** Run generated SQL through linter (pass) → execute on sandbox DB → check if results have expected columns and non-null values. If error: 'Column revenue not found in orders' → feed error back: 'Available columns in orders: id, user_id, total, created_at. Did you mean total?' → Regenerate.

Agent Tools via MCP (Model Context Protocol)

Each capability is exposed as a tool following the **MCP standard** (Anthropic's open protocol for tool interoperability): **schema_graph_tool** — input: table names; output: join paths from Neo4j. **rag_retrieval_tool** — input: NL question; output: relevant table DDLs + metadata. **sql_executor_tool** — input: SQL string; output: execution result or error. **sql_linter_tool** — input: SQL string; output: AST validity + column/table reference check. MCP ensures these tools can be consumed by other AI systems, IDE plugins, or external agents.

Agent Memory System

Three memory layers: **(1) Short-term (conversation buffer)** — stores the current session's query history and intermediate results in LangGraph's state. Cleared per session. **(2) Long-term (successful query cache)** — stores validated NL→SQL pairs in ChromaDB. When a similar question appears, the agent retrieves cached queries as few-shot examples, dramatically improving accuracy on repeated patterns. **(3) Episodic (error pattern store)** — logs failed queries with their error messages and corrections. The agent checks episodic memory before generating to avoid known mistakes (e.g., 'never use LEFT JOIN on the orders table when filtering by status').

5.6 Inference Engine

NL Preprocessor

Extracts intent (SELECT/aggregate/filter/group), entities (table and column references), and disambiguates references using the business glossary. Uses spaCy NER + custom rules for schema term linking. Example: 'show me active users who spent more than \$100 last month' → intent: SELECT with WHERE filter + aggregation, entities: users (table), orders.total (column, via glossary: 'spent' → orders.total), time_filter: 'last month' → WHERE created_at >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '1 month').

SLM Inference Runtime

The fine-tuned model (base + LoRA adapters) is served via **vLLM** for GPU deployment: vLLM's PagedAttention reduces KV-cache memory waste, continuous batching handles concurrent users, and LoRA adapter hot-loading enables serving multiple schema-specific adapters from a single base model. For CPU deployment: **llama.cpp** (GGUF quantization, Q4_K_M) or **ONNX Runtime** (optimized graph execution). Typical latency: 1-3 seconds (GPU), 5-15 seconds (CPU) per query.

5.7 SQL Validation & Self-Correction

Check	Tool	What It Catches	Action on Failure
1. Syntax	sqlglot parse()	Missing keywords, brackets, malformed clauses	Append syntax error to prompt, regenerate
2. Schema Refs	Custom validator	Non-existent table/column names, wrong aliases	Suggest closest matching column, regenerate
3. Join Logic	Graph validator	Missing ON conditions, cross-join without WHERE	Inject correct join path from graph, regenerate
4. Execution	Sandbox DB	Runtime errors, empty results, timeout (>10s)	Append DB error message, retry with fix
5. Sanity	Result checker	Unexpected NULL columns, zero rows, >1M rows	Flag as warning, ask user to confirm intent

5.8 Evaluation & Feedback Loop

Accuracy Metrics

Execution Accuracy (EX) — the primary metric. Runs both the generated SQL and the gold-standard SQL on the database, compares result sets. A query passes if the result sets are identical (order-insensitive). Target: >70%. **Exact Match (EM)** — secondary metric. Checks if the generated SQL string matches the gold standard after normalization (lowercase, whitespace standardization). **Valid SQL Rate (VSR)** — percentage of generated queries that parse without syntax errors. Target: >95%. **Join Accuracy** — custom metric. Percentage of queries where all JOIN conditions are correct. Critical for this project.

Active Learning Feedback Loop

Failed queries (EX=0) are logged with: NL question, generated SQL, error message, correct SQL (if provided by user). Periodically (weekly or after N accumulated corrections), the LoRA adapters are re-trained on the original training set PLUS the new corrected pairs. This creates a flywheel: more usage → more corrections → better model → fewer errors → more trust → more usage.

6. Technology Stack

Category	Technology	Version	Purpose
Base SLM	CodeLlama-7B-Instruct or Phi-3-mini-4k	Latest HF	Code-gen foundation model
Fine-Tuning	QLoRA (PEFT + TRL + bitsandbytes)	Latest	4-bit efficient fine-tuning
Graph DB	Neo4j Community	5.x	Schema graph + join paths
Vector Store	FAISS or ChromaDB	Latest	Schema embedding retrieval
Embeddings	all-MiniLM-L6-v2	Latest	384-dim sentence embeddings
Reranker	ms-marco-MiniLM-L-6-v2	Latest	Cross-encoder context scoring
Agent Framework	LangGraph (LangChain)	0.2+	Stateful agent orchestration
Tool Protocol	MCP	Latest	Tool interoperability standard
SQL Validation	sqlglot	Latest	AST parsing + transpilation
NL Processing	spaCy	3.x	NER + entity linking
Inference (GPU)	vLLM	0.6+	High-throughput LLM serving
Inference (CPU)	llama.cpp / ONNX RT	Latest	CPU-optimized inference
API Server	FastAPI	0.100+	REST API endpoints
Chat UI	Streamlit	1.30+	Business user interface
Guardrails	NeMo Guardrails	Latest	Prevent harmful SQL
Observability	LangSmith / Phoenix	Latest	Agent trace debugging
Monitoring	Prometheus + Grafana	Latest	Latency & accuracy metrics
Containerization	Docker + Compose	Latest	Reproducible deployment
Language	Python	3.11+	Primary implementation

7. Data Flow — End-to-End Walkthrough

This section traces a single user query through the entire system, showing exactly what happens at each stage.

Example Query: *'Show me total revenue per product category for users who signed up in 2024'*

Stage 1: NL Preprocessor

Extracts intent: SELECT with GROUP BY + WHERE filter. Entities: 'revenue' → orders.total (via glossary), 'product category' → categories.name, 'users' → users table, 'signed up in 2024' → users.created_at >= '2024-01-01'. Identified tables: users, orders, categories.

Stage 2: Graph Join Planner

Queries Neo4j: shortestPath(users → categories). Discovers: users → orders (users.id = orders.user_id) → order_items (orders.id = order_items.order_id) → products (order_items.product_id = products.id) → categories (products.category_id = categories.id). Returns 4-hop join chain with all ON conditions.

Stage 3: Vector Retriever

Embeds the NL question, searches FAISS. Returns top-5 tables: users, orders, order_items, products, categories — all with similarity scores > 0.7. Confirms the graph planner's table selection.

Stage 4: Context Assembler

Merges: CREATE TABLE DDLs for all 5 tables + JOIN path + business glossary mapping ('revenue' = SUM(orders.total)) + sample values. Reranker confirms all tables are relevant. Assembled prompt: ~1800 tokens of schema context.

Stage 5: SLM Generator

The fine-tuned CodeLlama-7B receives the prompt and generates: `SELECT c.name AS category, SUM(o.total) AS revenue FROM users u JOIN orders o ON u.id = o.user_id JOIN order_items oi ON o.id = oi.order_id JOIN products p ON oi.product_id = p.id JOIN categories c ON p.category_id = c.id WHERE u.created_at >= '2024-01-01' GROUP BY c.name ORDER BY revenue DESC`

Stage 6: Validator Agent

Check 1 (syntax): sqlglot parses successfully. Check 2 (schema refs): all table and column names exist in the graph. Check 3 (join logic): all ON conditions match graph FK edges. Check 4 (execution): runs on sandbox DB, returns 12 rows with non-null values. PASS.

Stage 7: Response

Returns the validated SQL to the user via FastAPI/Streamlit. Logs the successful NL→SQL pair to long-term memory for future few-shot retrieval.

8. Accuracy Strategy (70%+ Target)

Achieving 70%+ execution accuracy requires a layered approach. No single technique is sufficient — the combination is what reaches the target.

Technique	Expected Accuracy Gain	Cumulative
Base SLM (zero-shot)	35-45%	35-45%
+ Schema-conditioned fine-tuning	+15-20%	50-65%
+ RAG context retrieval	+5-8%	55-73%
+ Graph join path injection	+5-7%	60-80%
+ Agentic self-correction (3 retries)	+5-10%	65-85%
+ Few-shot from long-term memory	+3-5%	68-90%
+ Active learning re-training	+2-5% over time	70-95%

Figure 2: Accuracy contribution of each technique. Values are estimated ranges based on published benchmarks (Spider, BIRD) and empirical results from similar systems.

Key Insight: The agentic self-correction loop alone accounts for a 5-10% accuracy boost because it catches and fixes the 'easy' errors (wrong column names, missing JOINS) that the SLM generates on first attempt. This is the highest-ROI component to implement.

9. Deployment Architecture

Service	Container	Port	Resources
FastAPI Server	text-to-sql-api	8000	2 CPU, 4GB RAM
vLLM Inference	text-to-sql-vllm	8001	1 GPU (16GB+), 8GB RAM
Neo4j	text-to-sql-neo4j	7474/7687	2 CPU, 4GB RAM
ChromaDB	text-to-sql-chroma	8002	1 CPU, 2GB RAM
Streamlit UI	text-to-sql-ui	8501	1 CPU, 1GB RAM
Prometheus	monitoring-prometheus	9090	1 CPU, 1GB RAM
Grafana	monitoring-grafana	3000	1 CPU, 512MB RAM
Sandbox DB	text-to-sql-sandbox	5432	1 CPU, 2GB RAM

Deployment Options

Development: docker-compose with all services on a single machine with GPU. Total resources: 1 GPU (16GB+), 8 CPU cores, 24GB RAM. **Production:** Kubernetes cluster with: vLLM on GPU nodes (auto-scaling based on queue depth), Neo4j on persistent volume, ChromaDB on SSD-backed storage, API/UI on standard nodes with horizontal pod autoscaling. **Lite/CPU mode:** Replace vLLM with llama.cpp (GGUF Q4_K_M), reduce total to: 4 CPU cores, 16GB RAM, no GPU required. Latency: 5-15 seconds per query (acceptable for internal tools).

10. Project Timeline & Milestones

Week	Phase	Deliverables	Success Criteria
1-2	Schema Ingestion	Excel importer, SQL connector, schema normalizer	Can import any schema into canonical format
3-4	Graph Schema	Neo4j graph builder, join path discovery, Cypher queries	Discovers 3-hop join paths correctly
5-6	RAG Pipeline	FAISS indexing, dual retrieval, context assembler, reranker	Retrieves correct tables for 90%+ of queries
7-8	Fine-Tuning	Training data generation, QLoRA training, model eval	EX accuracy > 60% on test set
9-10	Agentic Layer	LangGraph agents, tool integration, self-correction	EX accuracy > 70% with retries
11-12	Integration & Deployment	FastAPI, Streamlit, Docker, guardrails, monitoring	End-to-end demo with live database
13-14	Testing & Documentation	Evaluation report, active learning loop, final docs	70%+ EX on 3 different schemas

11. Risks & Mitigation

Risk	Impact	Likelihood	Mitigation
Accuracy below 70%	High	Medium	Add more training data, increase retry count, use teacher LLM for difficult queries
GPU unavailable for training	Medium	Low	Use Google Colab Pro/A100, or train on cloud (Lambda, RunPod) for ~\$2/hr
Neo4j complexity for small schemas	Low	Medium	Fall back to NetworkX for schemas <200 tables
vLLM inference latency too high	Medium	Low	Use speculative decoding, reduce max_tokens, cache frequent queries
Schema changes break model	High	Medium	Auto-detect schema drift, trigger re-indexing + re-training pipeline
Harmful SQL generation	High	Low	NeMo guardrails block DDL/DML statements, read-only DB connections

12. References

- [1] Yu, T. et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP.
- [2] Li, J. et al. (2024). BIRD: A Big Benchmark for Large-Scale Database Grounded Text-to-SQL Evaluation. NeurIPS.
- [3] Roziere, B. et al. (2023). Code Llama: Open Foundation Models for Code. Meta AI.
- [4] Abdin, M. et al. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. Microsoft Research.
- [5] Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized Language Models. NeurIPS.
- [6] Kwon, W. et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP.
- [7] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.
- [8] LangGraph Documentation — <https://langchain-ai.github.io/langgraph/>
- [9] Neo4j Graph Data Science Library — <https://neo4j.com/docs/graph-data-science/>
- [10] Model Context Protocol (MCP) — <https://modelcontextprotocol.io/>
- [11] vLLM Project — <https://vllm.ai/>
- [12] Anthropic. (2025). Building Effective Agents. Anthropic Research.

End of Document — Capstone Project: Agentic Text-to-SQL SLM System